

Software for Computational Geometry

1 Introduction and Overview

- computational experiments
- overview of software packages

2 Getting Started

- cloud and/or local computing
- selecting package managers

3 Planar Convex Hulls

- V-representations and H-representations with SageMath
- the spatial package of scipy
- the cgal-swig-bindings
- running CGAL C++ examples

MCS 481 Lecture 3
Computational Geometry
Jan Verschelde, 28 August 2026

Software for Computational Geometry

1 Introduction and Overview

- computational experiments
- overview of software packages

2 Getting Started

- cloud and/or local computing
- selecting package managers

3 Planar Convex Hulls

- V-representations and H-representations with SageMath
- the spatial package of scipy
- the cgal-swig-bindings
- running CGAL C++ examples

computational experiments

MCS 481 is a computational course, we use software

- 1 to make and study interesting examples, and
- 2 to experimentally study the behavior of algorithms.

The purpose of this lecture is also twofold:

- 1 to present an overview of important software, and
- 2 to provide practical hints on getting started.

We distinguish between two types of software:

- 1 what is available in Python, conveniently wrapped, and
- 2 C++ code, which requires compiling test programs.

All software used in MCS 481 is free and open source.

Software for Computational Geometry

1 Introduction and Overview

- computational experiments
- overview of software packages

2 Getting Started

- cloud and/or local computing
- selecting package managers

3 Planar Convex Hulls

- V-representations and H-representations with SageMath
- the spatial package of scipy
- the cgal-swig-bindings
- running CGAL C++ examples

Overview of Software Packages

- 1 C. Bradford Barber, David P. Dobkin, and Hannu Huhdanpaa.
The Quickhull algorithm for convex hulls.
ACM Trans. Math. Softw. 22(4):469-483, 1996.
Available in `scipy.spatial`, url: www.qhull.org.
- 2 The ***Computational Geometry Algorithms Library*** (cgal.org)
provides reliable geometric algorithms as a C++ library.
CGAL provides an extensive collection of examples.
Its development is funded by academia and companies.
- 3 Jonathan Richard Shewchuk, ***Triangle: Engineering a 2D Quality Mesh Generator and Delaunay Triangulator.***
In *Applied Computational Geometry: Towards Geometric Engineering*,
edited by Ming C. Lin and Dinesh Manocha, volume 1148 of LNCS,
pages 203-222, Springer-Verlag, Berlin, May 1996.
Award: 2003 James H. Wilkinson Prize for Numerical Software.

Software for Computational Geometry

1 Introduction and Overview

- computational experiments
- overview of software packages

2 Getting Started

- cloud and/or local computing
- selecting package managers

3 Planar Convex Hulls

- V-representations and H-representations with SageMath
- the spatial package of scipy
- the cgal-swig-bindings
- running CGAL C++ examples

cloud and/or local computing

We will be using mainly Python as the scripting language and the Jupyter notebook to document our computations.

- 1 Collaborative Calculation and Data Science (cocalc.com) provides a cloud service to SageMath, through Jupyter notebooks.

SageMath uses Python as its main scripting language and provides many packages and computer algebra systems.

- 2 Install and run software on your own laptop and/or desktop.

Keep your operating system up to date,
keep track of the software you install, and
have enough free disk space.

Software for Computational Geometry

1 Introduction and Overview

- computational experiments
- overview of software packages

2 Getting Started

- cloud and/or local computing
- selecting package managers

3 Planar Convex Hulls

- V-representations and H-representations with SageMath
- the spatial package of scipy
- the cgal-swig-bindings
- running CGAL C++ examples

selecting package managers

Software installation can happen in two ways:

- 1 directly from source, downloaded via `github.com`, or
- 2 via a package manager.

Installing from source gives the latest version of the software, for our purposes, what is available via package managers suffices.

- For Python and the Jupyter notebook, use `conda`, and in particular `miniconda` to install only what is really needed.
- All Linux systems have their package managers.
- For MacOS computers, there is `brew`.
- Windows offers Ubuntu as `WSL`, Window Subsystem for Linux.

Software for Computational Geometry

1 Introduction and Overview

- computational experiments
- overview of software packages

2 Getting Started

- cloud and/or local computing
- selecting package managers

3 Planar Convex Hulls

- V-representations and H-representations with SageMath
- the spatial package of scipy
- the cgal-swig-bindings
- running CGAL C++ examples

V-representation and H-representation

Every (convex) polygon can be defined in two ways:

- 1 as the convex hull of a set of points: the *V-representation*,
- 2 as the intersection of half planes: the *H-representation*.

The convex hull problem computes both the V-representation and the H-representation of a set of points.

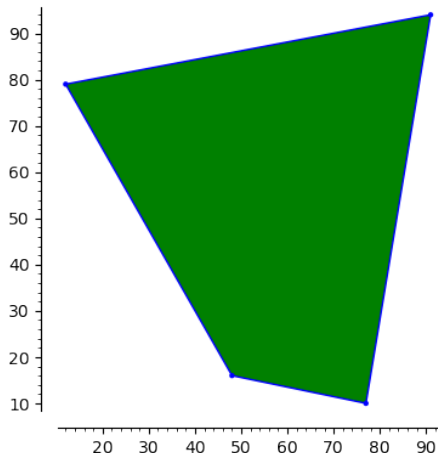
In SageMath, the representations can be computed quickly.

```
L = [[48, 16], [77, 10], [47, 47], [60, 27], [12, 79],  
     [36, 53], [91, 94], [28, 73], [41, 77], [70, 37]]  
P = Polyhedron(vertices=L)  
P.plot()
```

to be executed in a Jupyter notebook with SageMath kernel

the convex hull of ten points

as the output of P.plot()



V-representation and H-representation example

```
L = [[48, 16], [77, 10], [47, 47], [60, 27], [12, 79],  
      [36, 53], [91, 94], [28, 73], [41, 77], [70, 37]]  
P = Polyhedron(vertices=L)
```

```
P.Vrepresentation()
```

```
(A vertex at (12, 79),  
 A vertex at (91, 94),  
 A vertex at (77, 10),  
 A vertex at (48, 16))
```

```
P.Hrepresentation()
```

```
(An inequality (7, 4) x - 400 >= 0,  
 An inequality (6, 29) x - 752 >= 0,  
 An inequality (-6, 1) x + 452 >= 0,  
 An inequality (15, -79) x + 6061 >= 0)
```

Software for Computational Geometry

1 Introduction and Overview

- computational experiments
- overview of software packages

2 Getting Started

- cloud and/or local computing
- selecting package managers

3 Planar Convex Hulls

- V-representations and H-representations with SageMath
- **the spatial package of scipy**
- the cgal-swig-bindings
- running CGAL C++ examples

the spatial package of scipy

The core of Python's computational ecosystem consists of

- `numpy`, for array processing and linear algebra,
- `scipy`, for scientific computing,
- `matplotlib`, for making plots.

The `spatial` package of `scipy` for spatial algorithms and data structures: queries, Delaunay triangulations, convex hulls, and Voronoi diagrams.

The `ConvexHull` of `scipy.spatial` computes the convex hull with the Qhull library, see <http://www.qhull.org>.

the convex hull of 20 random points in the plane

the script random2dhull.py

```
import numpy as np
from scipy.spatial import ConvexHull
from scipy.spatial import convex_hull_plot_2d
from matplotlib import pyplot as plt

points = np.random.rand(20, 2)
hull = ConvexHull(points)
print('The vertices :')
print(hull.vertices)
print('The edges :')
for simplex in hull.simplices:
    print(simplex)
print('the equations :')
print(hull.equations)
convex_hull_plot_2d(hull)
plt.show()
```


output of the script

In a terminal window, type `python random2dhull.py`, which then shows:

The vertices :

```
[ 3  5  6 12  1 18 16]
```

The edges :

```
[3 5]
```

```
[18  1]
```

```
[12  1]
```

```
[16  3]
```

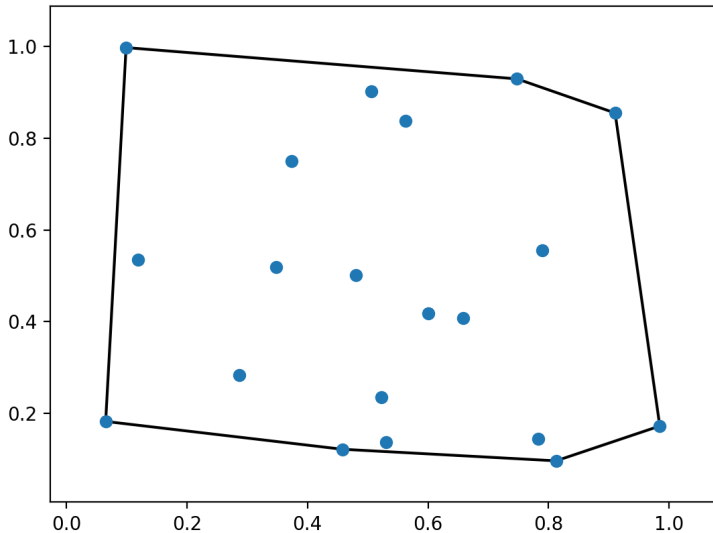
```
[16 18]
```

```
[ 6  5]
```

```
[ 6 12]
```

omitting the equations, the plot is on the next slide.

the plot of a convex hull in the plane



Software for Computational Geometry

1 Introduction and Overview

- computational experiments
- overview of software packages

2 Getting Started

- cloud and/or local computing
- selecting package managers

3 Planar Convex Hulls

- V-representations and H-representations with SageMath
- the spatial package of scipy
- **the cgal-swig-bindings**
- running CGAL C++ examples

the cgal-swig-bindings

Install the Python interface to CGAL with `pip install cgal`, or via `conda install -c conda-forge cgal`.

After a successful installation, in a Python prompt:

```
>>> import CGAL
>>> CGAL.__version__
'5.6.1'
```

The Python interface is still recent:

CGAL Made More Accessible, by Nir Goren, Efi Fogel, and Dan Halperin, [arxiv:2202.13889v2](https://arxiv.org/abs/2202.13889v2) [cs.CG] 7 Jun 2023.

The Python interface offers (almost) everything we need in this course.

script copied from the documentation

```
from __future__ import print_function
from CGAL.CGAL_Kernel import Point_2
from CGAL import CGAL_Convex_hull_2

L = []
L.append(Point_2(0, 0))
L.append(Point_2(1, 0))
L.append(Point_2(0, 1))
L.append(Point_2(1, 1))
L.append(Point_2(0.5, 0.5))
L.append(Point_2(0.25, 0.25))
print('the input points :')
for p in L:
    print(p)
result = []
CGAL_Convex_hull_2.convex_hull_2(L, result)
print('the vertices :')
for p in result:
    print(p)
```

the output

```
$ python convex2dcgal.py
```

```
the input points :
```

```
0 0
```

```
1 0
```

```
0 1
```

```
1 1
```

```
0.5 0.5
```

```
0.25 0.25
```

```
the vertices :
```

```
0 0
```

```
1 0
```

```
1 1
```

```
0 1
```

Software for Computational Geometry

1 Introduction and Overview

- computational experiments
- overview of software packages

2 Getting Started

- cloud and/or local computing
- selecting package managers

3 Planar Convex Hulls

- V-representations and H-representations with SageMath
- the spatial package of scipy
- the cgal-swig-bindings
- running CGAL C++ examples

running CGAL C++ examples

Installed CGAL 5.4 via the package manager of Ubuntu

```
sudo apt-get install libgmp-dev  
sudo apt-get install libmpfr-dev  
sudo apt-get install libcgall-dev
```

To check the version with the code `version.cpp` below:

```
#include <CGAL/version_macros.h>  
#include <iostream>  
  
using namespace std;  
  
int main()  
{  
    cout << CGAL_VERSION_STR << endl;  
    return 0;  
}
```

Compile as follows: `g++ version.cpp -o version.`

the example array_convex_hull_2.cpp

```
#include <iostream>
#include <CGAL/Exact_predicates_inexact_constructions_kernel.h>
#include <CGAL/convex_hull_2.h>

typedef CGAL::Exact_predicates_inexact_constructions_kernel K;
typedef K::Point_2 Point_2;

int main()
{
    Point_2 points[5] =
    { Point_2(0,0), Point_2(10,0), Point_2(10,10), Point_2(6,5), Point_2(4,1) };

    Point_2 result[5];

    Point_2 *ptr = CGAL::convex_hull_2( points, points+5, result );

    std::cout << ptr - result << " points on the convex hull:" << std::endl;

    for(int i = 0; i < ptr - result; i++)
    {
        std::cout << result[i] << std::endl;
    }
    return 0;
}
```

compiling and running

At the command prompt we can type:

```
g++ array_convex_hull_2.cpp -lgmp \  
-o array_convex_hull_2
```

Observe the linking with the GMP library.

The output of `/array_convex_hull_2` is

```
3 points on the convex hull:  
0 0  
10 0  
10 10
```

summary

We can compute planar convex hulls as follows:

- 1 Use SageMath, in the cloud at `cocalc.com`.
- 2 Use the `spatial` package of `scipy`.
- 3 Use CGAL with `swig-cgal-bindings`.
- 4 Use CGAL compiling C++ test programs.